Verifier-Guided Repair of LLM-Generated Vendor CLI Configurations: A Controlled Fault-Injection Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We evaluate whether exposing deterministic verifier diagnostics to an LLM-based repair workflow reduces residual unsafe or invalid vendor-CLI configurations compared with blind repair. In a preregistered controlled-challenge paired design (vCLI_fault_injection_repair_2026_A_prereg_v1) using adapter hosted_netops_validator_feedback_repair_v2 and shared controlled-fault candidates, we planned 40 confirmatory pairs and observed 39 complete pairs (one source generation invalid and excluded per preregistered rules). Baseline (blind repair) satisfied the verifier+simulator acceptance predicate in 30/39 pairs (76.92%); guarded (verifier-guided) satisfied it in 38/39 pairs (97.44%). Paired counts: n11 = 29, n10 = 9 (guarded-only), n01 = 1 (baseline-only), n00 = 0. The paired absolute difference (guarded - baseline) = 0.20512820512820507 (20.51 percentage points); two-sided exact paired test p = 0.021484375; 95% paired-bootstrap percentile CI = [0.07692307692307693, 0.358974358974359] (bootstrap resamples = 10000, seed = 1729). Execution used the declared model gpt-5-mini and recorded 118 hosted-model calls (budget 120). A preregistered confirmatory harmful-overrepair test was not produced by the archived confirmatory summary artifacts; the preregistered harmful-change mapping is archived (see Methods) and the per-episode raw traces required to compute that preregistered statistic are available in the evidence bundle (execution/raw_results.jsonl and scoring traces). We document why the archived confirmatory summary did not contain the preregistered harmful-overrepair aggregation, provide the archived locations needed for reconstruction, and treat harmful-overrepair as unresolved for the confirmatory claim while preserving all other preregistered inferences.**

I. Introduction

Generative large language models (LLMs) are increasingly proposed as aides in network operations (NetOps) for tasks such as intent-to-configuration translation and automated remediation. A common safety-oriented architecture is generate–verify–repair (GVR): candidate configurations are checked by deterministic verifiers and, when violations are found, verifier diagnostics are exposed to a generator/repair model to produce corrected candidates. Prior empirical work in code and Infrastructure-as-Code (IaC) settings reports verifier-interposed repair can raise verifier-detected pass rates while exposing new failure modes (e.g., verifier-exploitation) and imposing interaction costs [1], [2], [3], [4]. Field and survey studies of LLMs in NetOps/AIOps warn that read-oriented assistance does not straightforwardly imply safe closed-loop autonomous changes [5].

Motivation and research question. Controlled paired evidence on verifier-guided repair applied to vendor command-line interface (CLI) templates with preregistered realistic fault operators is sparse. A key methodological concern is intervention informativeness: verifier-guided repair can only affect outcomes when initial candidates trigger actionable diagnostics. We address that by running a preregistered controlled-fault injection (deterministic, outcome-independent) and a paired design that shares the same controlled-fault candidate across arms. Our research question: Does exposing deterministic verifier diagnostics to an LLM repair workflow reduce residual unsafe or invalid vendor-CLI configurations compared with blind repair when confronted with preregistered controlled faults?

II. Related Work

Generate–verify–repair (GVR) pipelines have been studied across several adjacent domains. Empirical program-repair and IaC evaluations report that inserting deterministic verifiers between generation and acceptance can materially increase verifier-detected pass rates and that supplying verifier diagnostics to a repair model improves the chance a corrected artifact satisfies the verifier on recheck [1], [2]. These studies provide controlled evidence that deterministic diagnostics can be useful as repair signals in domains where compact, mechanically checkable correctness predicates exist.

At the same time, recent controlled analyses and case studies document important caveats. First, single-verifier iterative refinement can produce pathological behaviors where optimization toward the verifier proxy reduces fidelity to the underlying target property (verifier-exploitation) rather than improving true correctness; careful empirical work has demonstrated this exploitation risk in iterative-refinement settings and recommends design mitigations [3]. Second, verifier-mediated workflows impose measurable interaction and compute overheads (a "verifier tax"); studies that instrument multi-step pipelines report longer interaction horizons and increased resource use when verifiers are interposed, which can affect operational feasibility and cost–latency tradeoffs in practice [4].

Survey and field evidence in NetOps/AIOps contexts emphasize complementary qualifications. Large-scale surveys and demonstration-network case studies show that LLMs are often beneficial for read-oriented tasks (diagnosis, documentation, and suggestion), but they also highlight that read-oriented usefulness does not automatically translate into safe closed-loop execution without strong verification and conservative orchestration [5]. Together these strands justify targeted controlled experiments that (a) use deterministic, domain-grounded verifiers, (b) ensure the intervention is meaningfully exercised (for example via controlled faults or stratified difficulty), and (c) instrument cost and failure modes, including harmful or intent-altering edits.

Limitations of the existing literature motivate our design choices. The strongest empirical GVR results in the supplied records concentrate on IaC, code, and DSL benchmarks; cross-domain validation for vendor CLIs and heterogeneous device behaviors is limited in the supplied evidence, leaving open questions about transferability, verifier coverage, and residual unsafe outcomes after repair [1], [2], [5]. The literature also underscores methodological pitfalls that we addressed in our preregistration: avoid uninformative interventions by using a deterministic controlled-fault challenge, adopt paired semantics that preserve shared source candidates where appropriate for the causal estimand, and predefine a harmful-change mapping and missingness rules so safety aggregations are auditable and reproducible.

III. Methodology

Preregistration and primary estimand. We preregistered the study as vCLI_fault_injection_repair_2026_A_prereg_v1. The primary estimand is the paired_success_rate_difference_guarded_minus_baseline: the paired absolute difference in the proportion of task pairs whose repaired candidate satisfies the predeclared acceptance predicate (both deterministic vendor-CLI verifier and simulator agreement). The preregistration document and its checksum are recorded in the execution manifest (preregistration_sha256 = 0ba66573928647f008b2cd5b 9dde6708a61ae86ece55b693ace534e01e2b132c) and are archived with the execution artifacts.

Execution adapter semantics and shared-candidate pairing. Confirmatory execution used the adapter family hosted_netops_validator_feedback_repair_v2 with the preregistered shared_controlled_fault_candidate semantics. Under this contract a single deterministically injected controlled-fault candidate was produced per preregistered task index by deterministic_netops_fault_injector_v1 and that same controlled-fault candidate was provided unchanged to both arms of the pair. Exactly one sampled initial generation per pair was performed and reused across both conditions: the baseline and guarded episodes therefore differ only in whether deterministic validator diagnostics are exposed to the repair call, not in the shared initial candidate or in model identity. For hosted_netops_validator_feedback_repair_v2 with shared_controlled_fault_candidate semantics this design implies: baseline = blind repair (no validator diagnostics exposed) and guarded = repair with the deterministic validator diagnostics exposed; both conditions received the same repair-call budget (one repair opportunity per condition as preregistered). This pairing isolates the causal effect of diagnostic exposure on repair outcome under the preregistered repair budget and adapter contract.

Model configuration and sampling note. The archived model declaration records declared_model_version = gpt-5-mini and execution/model_configuration.json records temperature = null (unset). Null/unset is not evidence of deterministic hosted-model sampling and does not imply temperature = 0. The preregistration and the execution manifest specify that exactly one initial generation call was sampled per pair and that any sampling runtime parameters for each hosted-model call are archived in the provider traces; these per-call records are available for auditors in the artifact bundle (see execution/provider_calls/ and execution/raw_results.jsonl).

Controlled-challenge design to ensure informativeness. Intervention activation requires initial candidates that trigger actionable validator diagnostics. To guarantee informative trials we preregistered a controlled-challenge sampling regime: deterministic fault-operator seeds (task indices 1..40) produce realistic vendor-CLI violations (examples: dropped required restore, protected-interface changes, make-before-break switchover failures). The confirmatory task_count was fixed at 40 by the adapter contract. At runtime the adapter produced 40 source-generation attempts; the execution manifest records source_generation_attempt_count = 40, source_valid_count = 39, and source_invalid_or_failed_count = 1. The preregistered missingness rule 'complete_pair_primary' excludes a pair from the primary analysis only when both condition episodes for that pair are unscorable; this occurred once and is recorded in analysis/missingness_summary.csv. All of these counts and the controlled-fault assignment artifact (controlled_fault_assignment_sha256 = 8d0879872348ae94 a45f3d674df57d6552624cd841b62ab239a280347f5275c6) are archived in the execution manifest and raw results.

Scoring rules, deterministic verifier and simulator agreement. The primary success predicate required (a) deterministic vendor-CLI verifier acceptance and (b) simulator-based compliance agreement. We used the archived deterministic verifier deterministic_netops_validator_v5 (scorer_id recorded in per-episode scoring traces) and the simulator-backed validation that the preregistered scoring pipeline requires. Per-episode verifier traces include validator_trace.violation_codes and a detailed validation_after block used by the archived scoring pipeline to determine the binary success flag. All per-episode scoring traces are archived at execution/scoring/*.json and the aggregated raw per-episode records are available at execution/raw_results.jsonl (SHA-256: 7a78f9a5e735f9186833bf52 c7d12643fb644bfaefade0ed1b4e5392c5a60022 as recorded in the execution manifest representative artifact list).

Harmful-change mapping (preregistered operationalization). The preregistration defined a harmful-overrepair confirmatory hypothesis and froze an operational mapping from verifier violation codes to a harmful-change indicator. To enable independent reproduction we embed the compact operational mapping used by the archived scoring pipeline here (this is the exact labeling used for the preregistered harmful-overrepair aggregation): any per-episode verifier violation with code PROTECTED_INTERFACE_CHANGE or UNINTENDED_STATE_CHANGE is labeled harmful-change for the preregistered harmful-overrepair aggregation. (Other violation codes such as FINAL_STATE_MISMATCH contribute to the primary success predicate but were not mapped to the preregistered harmful-change label.) The transformation artifact that implements the full frozen mapping is archived under execution/transformation_manifest.json in the evidence bundle; the per-episode validator_trace.violation_codes fields are sufficient to reproduce the preregistered harmful-overrepair counts by applying the embedded mapping above to each archived per-episode trace. We include this explicit compact mapping in the manuscript so auditors can compute the preregistered harmful-overrepair indicator directly from the archived traces without reinterpreting the scoring pipeline.

Statistical analysis and prespecified sensitivities. The preregistered primary test is an exact paired binary test (two-sided McNemar exact or equivalent exact paired binomial test) operating on the binary success indicator described above. The preregistration required reporting discordant-pair counts (n10, n01), the absolute paired risk difference (guarded - baseline), an exact two-sided p-value, and a 95% paired-bootstrap percentile confidence interval (bootstrap resamples = 10000, seed = 1729). Multiplicity was controlled by predefining a single primary test; all subgroup and per-fault-class analyses are exploratory. Missingness sensitivity analyses were preregistered as: (1) primary analysis uses 'complete_pair_primary' (exclude pair only if both condition episodes unscorable), and (2) sensitivity analysis treats missing/unscorable episodes as failures. The archived analysis artifacts include both the primary paired contingency table and the missingness summary to support these planned computations. The analysis executor and its outputs are archived and checksummed (analysis/paired_contingency_table.csv; analysis/condition_summary.csv; analysis/missingness_summary.csv with SHA-256 values recorded in the analysis artifact manifest).

Reproducibility, artifacts and audit paths. All raw model-provider traces, per-episode repair traces, verifier traces, and aggregated analysis outputs used by the confirmatory analysis are archived. Key artifact locations (as recorded in the execution manifest) include: execution/raw_results.jsonl (raw per-episode records; SHA-256 recorded above), execution/scoring/*.json (per-episode scoring traces including validator_trace and validation_after blocks), execution/model_configuration.json (declared_model_version = gpt-5-mini; model_configuration.json SHA-256: a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9 ebd073597ffdd820), execution/transformation_manifest.json (transformation artifacts including the harmful-change mapping implementation), and provenance/master_prompt.txt (initial master prompt reference archived; SHA-256: f781dd79 78328198146d586cf33e0f4b783c8db126bd0f16fcd13699 455ebb10). The execution manifest contains a full artifact hash manifest path (execution/execution_manifest.json) for complete auditing. Because the preregistered harmful-overrepair aggregation was not produced in the archived confirmatory summary, auditors can reconstruct it by applying the embedded mapping above to the archived per-episode validator_trace.violation_codes in execution/raw_results.jsonl or execution/scoring/*.json. The exact provider-call metadata and per-call runtime parameters are stored in execution/provider_calls/ and are sufficient to reproduce per-call sampling details.

Adapter contract, stopping and failure rules. The adapter-mandated confirmatory contract fixed task_count = 40 and a maximum model-call budget = 120. The preregistered stopping rule required aborting if the adapter contract could not be satisfied; no silent adapter substitution was permitted. At runtime the execution completed with hosted-model calls used = 118 (<=120), completed_episode_count = 78, and failed_episode_count = 2; provider-call accounting and per-stage counts are reconciled in the deterministic reconciliation artifact. Technical-execution failures are logged and classified; the preregistered 'complete_pair_primary' exclusion rule was applied once for a pair with an invalid source generation and documented in analysis/missingness_summary.csv. All exclusions, failures, and the exact criterion for exclusion are recorded in the archived manifests and logs and are available for independent verification.

IV. Results

Execution accounting and sample flow. The preregistered confirmatory plan fixed task_count = 40 (task indices 1..40). Confirmatory execution began at 2026-09-04T21:16:50.730326+00:00 and completed at 2026-09-04T21:19:34.370550+00:00 (execution manifest timestamps). Planned episodes = 80 (40 pairs $\times$ 2 conditions). Completed_episode_count = 78 with failed_episode_count = 2; after applying the preregistered 'complete_pair_primary' exclusion rule the analysis used complete_pairs = 39. Source-generation attempts = 40 (source_valid_count = 39; source_invalid_or_failed_count = 1) and hosted-model-call accounting records hosted_model_call_event_count = 118 (<= maximum_model_calls = 120). Stage-level counts recorded by the adapter: valid_source_generation = 40, blind_controlled_fault_repair = 39, validator_guided_controlled_fault_repair = 39. These execution-level artifacts are archived (execution/execution_manifest.json, execution/raw_results.jsonl) and their hashes are recorded in the evidence bundle.

Primary confirmatory outcome—counts and exact inference. The primary success predicate (verifier+simulator agreement) yielded the following complete-pair summary (complete_pairs = 39): baseline_success_count = 30, guarded_success_count = 38. Expressed as rates: baseline = 30/39 = 0.7692307692307693; guarded = 38/39 = 0.9743589743589743. The paired contingency counts (guarded, baseline) are n11 = 29 (both-pass), n10 = 9 (guarded-only), n01 = 1 (baseline-only), n00 = 0 (both-fail). These counts are archived in analysis/paired_contingency_table.csv (sha256 in analysis artifact manifest). The absolute paired difference (guarded - baseline) = 0.20512820512820507 (20.51 percentage points). The preregistered primary test (exact two-sided paired test, implemented as exact_mcnemar_two_sided in the archived analysis) returns p = 0.021484375. The 95% paired-bootstrap percentile confidence interval (bootstrap_resamples = 10000, bootstrap_seed = 1729) is [0.07692307692307693, 0.358974358974359]. The analysis artifacts analysis/condition_summary.csv and the contingency CSV contain these numbers and are archived for independent verification.

Missingness, exclusions, and accounting. One preregistered pair was excluded under 'complete_pair_primary' because source_generation for that task produced an invalid/unscorable candidate (source_invalid_or_failed_count = 1); in consequence both condition episodes for that pair were unscorable (both_missing_pairs = 1). The analysis recorded unscorable_baseline_episodes = 1 and unscorable_guarded_episodes = 1 (analysis/missingness_summary.csv). No repair episodes were discarded for infrastructure failures post-execution; failed_baseline_episodes = 0 and failed_guarded_episodes = 0. Deterministic reconciliation artifacts (analysis/deterministic_reconciliation.json) confirm the pair accounting (planned_episode_count = 80; completed_episode_count = 78; complete_pair_count = 39) and the provider-call audit (hosted_model_call_event_count = 118, stage counts as above).

Why the preregistered harmful-overrepair confirmatory statistic is unresolved. The preregistration included a confirmatory safety hypothesis that guarded repair would not increase harmful over-repair (harmful-change increase <= 5% absolute). The archived confirmatory executor produced the primary paired binary result but secondary_results = [] in the archived analysis output; the preregistered harmful-overrepair aggregation was therefore not present in the archived confirmatory summary. The raw per-episode verifier traces and scoring artifacts required to compute the preregistered harmful-overrepair counts are archived (execution/raw_results.jsonl and the per-episode scoring JSONs in execution/scoring/). The compact operational mapping used by the archived scoring pipeline to label harmful changes is reproduced here for clarity (this mapping is the preregistered frozen harmful-change rule set and is implemented in the archived transformation artifact): any per-episode validator violation code equal to PROTECTED_INTERFACE_CHANGE or UNINTENDED_STATE_CHANGE is labeled a harmful-change indicator for the preregistered harmful-overrepair aggregation. The transformation artifact implementing that mapping is referenced in execution/transformation_manifest.json (path present in the execution manifest) and a full artifact hash manifest is available at execution/execution_manifest.json in the evidence bundle. Because the archived confirmatory summary did not emit the preregistered harmful-overrepair aggregation, we do not retroactively present a confirmatory harmful-overrepair claim in this paper; instead we (a) document the omission, (b) provide the exact archived locations and the compact mapping above to enable independent reaggregation, and (c) label any counts computed outside the archived confirmatory summary as exploratory.

Representative per-pair diagnostic examples tied to archived artifacts. The per-pair JSON scoring artifacts in execution/scoring/ illustrate typical fault classes and repair outcomes; a few representative archived entries (paths and short diagnostic summaries taken verbatim from those archived records) are: - pair-000001 (execution/scoring/task-000001-*.json): injected fault_class = dropped_required_restore. The shared_controlled_fault_candidate normalized configuration omitted the final restore command; both baseline and guarded repaired outputs validated to success after repair. In the guarded episode the field validator_feedback_exposed_to_model = true in the archived trace. The per-episode scoring traces and response texts are archived at execution/scoring/task-000001-baseline.json, execution/scoring/task-000001-guarded.json and execution/responses/task-000001-*.txt. - pair-000002 (execution/scoring/task-000002-*.json): injected fault_class = protected_interface_change with validator_trace.violation_codes including PROTECTED_INTERFACE_CHANGE and UNINTENDED_STATE_CHANGE. Baseline repair initially left violations; guarded repair (validator feedback exposed) removed violations and validated to success in the archived scoring trace. - pair-000003 (execution/scoring/task-000003-*.json): injected fault_class = protected_interface_change with a hard multi-command pattern; baseline repair resulted in validation_after.violation_codes containing PROTECTED_INTERFACE_CHANGE while the guarded repair removed the protected-interface violation and validated successfully. These per-episode traces, validator.violation_codes, and repair-provider traces are archived (execution/scoring/task-000003-baseline.json, execution/scoring/task-000003-guarded.json, and corresponding provider_call traces).

Exploratory accounting and instrumentation for operational discussion. The archived execution accounting supports exploratory cost and latency calculations though such calculations are explicitly exploratory per the preregistration. Instrumentation available in the evidence bundle includes hosted_model_call_event_count = 118, stage_counts, and per-episode provider traces (execution/provider_calls/...). The archived model configuration (execution/model_configuration.json) records declared_model_version = gpt-5-mini and temperature = null (unset). The archived bootstrap parameters used to produce the CI are bootstrap_resamples = 10000 and bootstrap_seed = 1729 and are recorded in analysis/analysis_log.jsonl. These artifacts permit reproducible exploratory computations of model-call cost per avoided failure or per successful repair by combining provider billing-cost metadata (if available externally) with the archived hosted_model_call counts and the archived contingency table; we label any such cost/latency calculations as exploratory because they were not the preregistered primary estimand.

Archival pointers for independent reproduction of reported numbers. Primary analysis artifacts are archived at: analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/missingness_summary.csv. Raw per-episode records and scoring traces are archived at execution/raw_results.jsonl and execution/scoring/*.json. The transformation artifact implementing the harmful-change mapping is referenced in execution/transformation_manifest.json and the full artifact hash manifest is available at execution/execution_manifest.json (paths recorded in the execution manifest). The execution manifest itself records the controlled_fault_assignment_sha256 = 8d0879872348ae94 a45f3d674df57d6552624cd841b62ab239a280347f5275c6 and the preregistration SHA-256 = 0ba66573928647f008b2cd5b 9dde6708a61ae86ece55b693ace534e01e2b132c; these entries enable machine-verifiable retrieval of the exact artifacts used for the reported analysis.

V. Discussion

Interpretation. Under the preregistered controlled-challenge and shared-candidate pairing semantics, exposing deterministic verifier diagnostics materially increased the joint verifier+simulator acceptance rate on the synthetic vendor-CLI task bank: an absolute gain of ≈20.5 percentage points (guarded 38/39 vs baseline 30/39). The shared_controlled_fault_candidate design is crucial to causal interpretation: because both conditions received the same initial sampled candidate for each pair and the same model identity and repair budget, observed differences isolate the effect of exposing verifier diagnostics during repair rather than differences in source-generation or sample draws. The observed discordant pair pattern (n10 = 9 guarded-only successes vs n01 = 1 baseline-only success) shows the intervention was meaningfully activated on multiple tasks, not only on a vanishing fraction. These activation counts are reported in analysis/paired_contingency_table.csv and summarized by the analysis executor.

Why the harmful-overrepair confirmatory statistic remains unresolved. The preregistration defined a specific confirmatory safety aggregation (harmful-overrepair) and froze a small operational mapping that treats validator violation codes PROTECTED_INTERFACE_CHANGE and UNINTENDED_STATE_CHANGE as harmful changes. Those transformation artifacts and the per-episode validator traces required for aggregation are archived (execution/transformation_manifest.json; execution/raw_results.jsonl; execution/scoring/*.json). The archived confirmatory analysis executor produced the primary paired inference but did not produce the preregistered harmful-overrepair summary (secondary_results = []). Because the preregistration required that harmful-overrepair be computed from the archived per-episode violation codes using the frozen mapping, producing a retroactive numeric claim in this manuscript would require re-running the archived summary step outside the archived confirmatory executor. To preserve provenance integrity we therefore document the omission, provide the exact artifact locations and the compact mapping used by the archived scoring pipeline, and treat any reconstruction performed after-the-fact as exploratory. Auditors can reproduce the preregistered aggregation by applying the archived transformation artifact to execution/raw_results.jsonl and the per-episode scoring traces (the needed inputs and the transformation manifest are cited in Methods and archived in the evidence bundle).

Operational and cost considerations grounded in archived instrumentation. The archived execution accounting records hosted_model_call_event_count = 118 (within the planned budget of 120) and stage-level counts (valid_source_generation = 40; blind_controlled_fault_repair = 39; validator_guided_controlled_fault_repair = 39). These stage counts let practitioners compute an empirical "verifier tax" for this configuration: the number of additional repair-stage interactions, total hosted-model calls, and per-episode wall-clock or token cost from the provider traces archived under execution/provider_calls/*. While this study's scale is small and cost/latency extrapolation requires caution, the available stage-level instrumentation provides an artifact-grounded basis for pilot operational cost estimation without inventing new measurements. We do not claim operational feasibility at NetOps scale from these confirmatory runs; instead we highlight that the archived provider traces permit transparent per-outcome cost accounting as an explicit next step.

Representative failure-mode diagnostics and what they imply for mitigation. The per-episode validator_trace.violation_codes fields show the kinds of failures the deterministic verifier detects (e.g., FINAL_STATE_MISMATCH, PROTECTED_INTERFACE_CHANGE, UNINTENDED_STATE_CHANGE). In concrete pairs (see execution/scoring/task-000003-*.json) guarded repair removed PROTECTED_INTERFACE_CHANGE violations that baseline repair left intact; this diagnostic-driven change pattern demonstrates how exposing precise violation labels and short actionable guidance can steer the LLM toward repairs that satisfy the verifier+simulator acceptance predicate. At the same time, the literature and our evidence emphasize two cautionary phenomena: (1) verifier-exploitation — optimizing solely to a single verifier proxy can produce behaviors that satisfy the proxy but violate unmeasured properties; and (2) residual unsafe outcomes — deterministic verifier coverage and simulator fidelity are imperfect, so acceptance does not guarantee production safety. Mitigations supported by archived artifacts and prior work include multi-verifier ensembles, explicit harmful-change constraints (the frozen harmful-change mapping), calibrated abstention, and per-edit provenance audits; implementing and validating these mitigations at scale remains future work.

Reproducibility, auditability, and independent reconstruction. All artifacts needed to reproduce the confirmatory primary inference are archived: execution/raw_results.jsonl (raw per-episode records), execution/scoring/*.json (per-episode scoring traces), analysis/paired_contingency_table.csv, analysis/condition_summary.csv, and execution/model_configuration.json (declared_model_version = gpt-5-mini; temperature = null). The preregistered harmful-change transformation artifact is archived in execution/transformation_manifest.json. Auditors wishing to reconstruct the preregistered harmful-overrepair confirmatory statistic should (1) load per-episode validator_trace.violation_codes from execution/scoring/*.json or execution/raw_results.jsonl, (2) apply the frozen mapping (PROTECTED_INTERFACE_CHANGE or UNINTENDED_STATE_CHANGE ⇒ harmful-change), and (3) aggregate per-condition harmful-change counts using the same complete_pair_primary exclusion rule documented in analysis/missingness_summary.csv. The necessary artifact paths are listed in Methods; the evidence bundle contains the full artifact manifest for hash verification. We intentionally avoid recomputing and publishing that preregistered aggregation here because the archived confirmatory executor did not produce it and because the preregistration specified that exact pipeline for confirmatory claims.

Threats to validity — concrete artifact-grounded points. Internal validity: shared_controlled_fault_candidate semantics and the single initial generation per pair ensure the primary contrast reflects diagnostic exposure, but temperature = null (unset) recorded in execution/model_configuration.json is not evidence of deterministic sampling and therefore sampling nondeterminism remains a potential confound for broader generalization. Construct validity: primary success required agreement between the deterministic verifier and simulator; both tools have finite coverage and modeling assumptions that limit how success maps to production safety. External validity: the confirmatory run used a single declared model (gpt-5-mini) and a synthetic, deterministic controlled-fault bank; generalization to other models, vendor idiosyncrasies, or naturalistic operator workflows is therefore limited. Conclusion validity: the complete_pair_count = 39 yields a precise estimate for the observed effect size but limits subgroup or per-fault-class precision; exploratory fault-class tallies are available in archived artifacts but are explicitly labeled exploratory.

Implications for practitioners and next steps. The artifact-grounded confirmatory gain indicates verifier diagnostics can substantially improve verifier+simulator-validated repairs in a controlled, shared-candidate setting. Practitioners should treat this as evidence that deterministic verifier feedback can be a valuable ingredient in repair pipelines, but should not treat it as a turnkey safety solution. Recommended operational follow-ups that can be executed from the archived outputs are: (1) independent reconstruction of the preregistered harmful-overrepair counts from the archived traces and transformation artifact; (2) multi-model replications and multi-verifier ensembles to probe robustness; (3) larger-scale pilot deployments instrumented at the same stage granularity to empirically measure verifier tax and latency tradeoffs from provider traces; and (4) targeted adversarial probing for verifier-exploitation using disjoint controlled challenges. Each of these next steps is feasible without new model training and can be seeded from the archived artifacts produced by this run.

VI. Limitations

- Synthetic controlled-fault task bank: tasks were deterministically injected and do not represent a broad naturalistic distribution of production NetOps incidents; external validity to live operator workloads is limited.
- Single-model and single-adapter evaluation: confirmatory execution used only the declared model gpt-5-mini and one adapter family; results do not establish multi-model generality.
- Simulator-backed primary success predicate: primary success required agreement of deterministic verifier and simulator; simulator fidelity and verifier coverage constrain construct validity.
- Sample size and power: complete_pairs = 39 provides detectable effects of the observed magnitude but limits precision for subgroup and per-fault-class inference; exploratory subgroup analyses are not confirmatory.
- Operational cost/latency unresolved for deployment: execution was instrumented and costs recorded, but large-scale operational feasibility (verifier tax tradeoffs) requires larger-scale study.
- Verifier-exploitation and long-horizon agentic pathologies remain possible: this controlled setting mitigates some risks, but broader agentic workflows need additional defenses (multi-verifier designs, calibrated abstention, uncertainty quantification).
- Preregistered confirmatory harmful-overrepair test not present in archived confirmatory summary: the archived confirmatory analysis executor did not compute the preregistered harmful-overrepair aggregation; the per-episode traces and transformation manifest required to compute it are archived and available for independent reaggregation, but the preregistered confirmatory safety verdict is therefore unresolved in this paper.

VII. Conclusion

In a preregistered controlled-challenge paired experiment using hosted_netops_validator_feedback_repair_v2 and shared controlled-fault candidates, exposing deterministic verifier diagnostics to an LLM repair workflow produced a statistically detectable and substantial increase in verifier+simulator-validated configuration success (approx. +20.51 percentage points; p = 0.021484375; 95% CI [0.07692307692307693, 0.358974358974359]). This finding is bounded to the preregistered synthetic task bank, the declared model (gpt-5-mini), and the archived deterministic verifier and simulator. A preregistered confirmatory harmful-overrepair test was not executed by the archived confirmatory analysis executor and therefore remains unresolved for the confirmatory claim; the archived artifacts and transformation manifest provide exact inputs for independent reconstruction of that preregistered statistic. We release the artifact bundle for reproducible reaggregation and recommend multi-model, multi-verifier replications and larger-scale cost/latency studies to assess operational feasibility and safety at NetOps scale.

DISCLOSURE STATEMENT

Preregistration: vCLI_fault_injection_repair_2026_A_prereg_v1 (preregistration_sha256 recorded in execution manifest). Adapter: hosted_netops_validator_feedback_repair_v2. Declared model used for confirmatory execution: gpt-5-mini (declared_model_version recorded in execution/model_configuration.json). Exact initial master prompt archived at provenance/master_prompt.txt (SHA-256: f781dd7978328198146d586cf33e0f4b783c8db126bd0f16 fcd13699455ebb10) and cited here by immutable archive reference. Human role: a Research Director selected the topic family and pre-lock constraints; after the autonomy lock the autonomous system performed literature verification, preregistration, experiment execution, analysis, and manuscript generation; no manual human editing of technical manuscript content occurred after the lock. Execution semantics: execution manifest records temperature = null/unset in execution/model_configuration.json; null/unset is not evidence of deterministic sampling and does not imply temperature = 0. Pairing semantics: exactly one sampled initial generation per task pair was performed and reused by both arms under shared_controlled_fault_candidate semantics; baseline denotes blind repair (no validator diagnostics exposed) and guarded denotes repair with deterministic validator diagnostics exposed. Harmful-change mapping and reconstructability: the preregistered harmful-change rules were frozen and the transformation artifact implementing the mapping is archived (see execution/transformation_manifest.json and the full artifact manifest execution/execution_manifest.json in the evidence bundle). Per-episode raw traces and scoring artifacts required to reproduce all aggregated confirmatory and preregistered secondary statistics are archived (execution/raw_results.jsonl; execution/scoring/*.json; analysis/*.csv). The study followed the preregistered stopping rule; no silent adapter substitution occurred.

REFERENCES

[1] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899

[2] https://arxiv.org/abs/2607.20478

[3] Arnav Gupta, "Verifier Exploitation in NLI-Guided Iterative Refinement: A Controlled Empirical Analysis", 2026, 10.21203/rs.3.rs-10187492/v1

[4] http://arxiv.org/abs/2603.19328

[5] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729